

プログラミング言語論

第4回

手続きの引数機構

変数の有効範囲

篠埜 功

変数名から値への対応

命令型言語において変数名は番地を表す。その番地へアクセスすると値が得られる。ある変数名に対する番地は、その変数の宣言によって割り当てられる。変数名から宣言、番地への対応は、言語の有効範囲の規則と引数機構によって決まる。

変数名の扱いについて、(プログラミング言語設計者にとって)いくつかの選択肢がある。

- 手続き呼び出し時の引数の受け渡し方法について
call by value, call by reference, call by name
- 変数と宣言の対応関係について
Static scope, dynamic scope

手続き(procedure)

手続きとは、プログラムの一部に名前を付けるものである。その部分のことを手続きの本体という。手続きが呼ばれると、手続きの本体が実行される。この手続きの本体の実行のことを手続きの*activation*という。関数(function)は、値を返す手続きのことである。関数、手続きという用語を区別をしないで用いる場合もある。

手続き、関数の呼び出し

関数呼び出しは式であり、手続き呼び出しは文である。

(例1) $r * \sin(\text{angle})$

$\sin(\text{angle})$ は関数呼び出し式である。式なので、式が書ける部分に自由に書くことができる。

(例2) $\text{read}(\text{ch})$

$\text{read}(\text{ch})$ は手続き呼び出し文である。文なので、文が書ける部分に自由に書くことができる。

手続き(関数)呼び出しの構文

手続き呼び出し文(関数呼び出し式)の構文(前置記法)

<手続き名> (<引数列>)

手続き呼び出し文(関数呼び出し式)における引数は実引数(actual parameter)と呼ばれる。

(例1) 関数呼び出し式 `sin (angle)` において `angle` は実引数。

(例2) 手続き呼び出し文 `read (ch)` において `ch` は実引数。

手続き(関数)呼び出しにおいて、引数が0個でも括弧が必須の場合が多い(C, Modula-2など)。Pascalでは引数がない場合は括弧を書かない。

(Pascalの例) **begin while** `eoln` **do** `readln`; `read(ch)` **end** において、`eoln`は引数無し関数呼び出し式、`readln`は引数無し手続き呼び出し文、`read(ch)`は引数有手続き呼び出し文。

手続きの宣言の構文

手続き(関数)の宣言の構文定義は、

- 手続きの名前
- 引数の名前と型
 - この引数は仮引数(formal parameter)という。
- 結果の型(関数の場合)
- 局所宣言と文の並び

の4つが明確になるように、(言語設計者が)設計する。

手続きの宣言の例(Pascal)

```
procedure getch;  
  begin  
    read (ch)  
  end;
```

これはgetchという名前の引数無し
の手続きの宣言である。

関数の宣言の例(Pascal)

```
function f (x : integer) : integer;  
var square : integer;  
begin  
    square := x * x;  
    f := square + 1  
end;
```

この例では、fという関数を宣言しており、引数はinteger型、返り値もinteger型であるということを表している。また、Pascalでは、関数名 := ...という代入文によって、関数の返り値が決まる。普通は、return文を用いる(C, Modula-2など)。

再帰関数

```
function f (n : integer) : integer;  
begin  
    if n = 0 then f := 1 else f := n * f (n-1);  
end;
```

これは、階乗を計算する関数である。

例えば、f (3) を計算するときは

$$f(3) = 3 * f(2) \qquad \qquad \qquad = 6$$

$$f(2) = 2 * f(1) \qquad \qquad \qquad = 2$$

$$f(1) = 1 * f(0) \qquad \qquad \qquad = 1$$

$$f(0) = 1$$

のように関数fが繰り返し呼び出される。

引数の渡し方について

```
function square (x : integer) : integer;  
begin  
    square := x * x  
end;
```

という関数宣言において、xは仮引数である。例えば、square(2)という関数呼び出し式の値は、xに2が代入された状態でx*xを評価した結果であり、4である。square(3)の値は9である。このように数を渡す場合は自明だが、変数や配列の要素を渡す場合はいくつかの方法がある。

引数の渡し方

引数の渡し方には、大きく分けて、

- Call-by-value (値呼び、値渡し)
- Call-by-reference (参照呼び、参照渡し)
- Call-by-name (名前呼び、名前渡し)

の3つがある。

Call by value

仮引数には実引数の評価結果の値が渡される。

手続き(関数) p の仮引数が x のとき、手続き(関数)呼び出し $p(e)$ の実行(評価)は

- (1) $x := e$

- (2) 手続き(関数)本体の実行

- (3) 手続き p が関数の場合は値を返す

という順で行われる。

(注) 変数 x が呼び出し側でも宣言されている場合、仮引数 x はそれとは別の変数である。

(squareの例) 関数square (2+3)の評価は、まず $x := 2 + 3$; が実行される。つまり2+3を評価し、その値5がsquareの仮引数 x に代入される。そののち $x * x$ が評価され、25が得られ、それが式square(2+3)の評価結果となる。

うまくいかない例 1

```
procedure nget (c : char);  
  begin  
    read (c)  
  end;
```

この例ではcにキーボードから読み込んだ値が入るが、例えば nget (ch) などと呼び出しても変数chには影響がない。(chとcは別の変数なので)

他の例として、swap (x,y)がある(変数xと変数yの値を入れ替える)

うまくいかない例2

```
procedure swap (x : integer; y : integer);  
var z : integer;  
  begin  
    z := x; x := y; y := z  
  end;
```

swap (a,b)のような呼び出しによって、変数aとbの値を入れ替えることはできない。

xとyには変数aとbの値が代入され、その後xとyの値が入れ替えられるので、変数a, bには影響が及ばない。

Call by reference

仮引数には実引数のアドレスが渡される。
(渡されたアドレスが仮引数のアドレスとなる。)

Pascalにはcall by referenceがあり、

```
procedure p (x : integer; var y : integer);
```

....

のように、call by referenceにしたい仮引数部分に
varをつけることによって、その仮引数の処理がcall
by referenceになることを表す。

pの第二引数には、変数や配列の要素など、アドレスを持つ式(代入文の左辺に書ける式)を与えなければならない。

swapの例(Pascal)

```
procedure swap (var x : integer; var y : integer);  
var z : integer;  
begin  
    z := x; x := y; y := z  
end;
```

この例では、xもyもcall by referenceである。例えば、swap (i, A[i])は以下のように実行される。

- (1) xのアドレスをiのアドレスにする。
- (2) yのアドレスをA[i]のアドレスにする。
- (3) z := x; x := y; y := z

例えば iが2, A[2]が99の場合、z := 2; i := 99; A[2] = z
の実行と同じ効果を持つので、iとA[2]の値が入れ替わることが分かる。

練習問題

以下のPascal プログラムを実行したときの出力結果を示せ。
手続きの仮引数にvarがついている場合、call by reference であることを表す。writeln は引数の値を出力後改行する。

```
program test;  
var x : integer;  
var y : integer;  
procedure swap  
  (var x: integer; var y : integer);  
var z : integer;  
begin  
  z := x; x := y; y := z  
end;
```

```
begin  
x := 3;  
y := 4;  
swap (x,y);  
writeln (x);  
writeln (y)  
end.
```

C言語について

C言語での引数の受け渡しはcall by valueのみである。
ただ、Cにはポインタがあり、ポインタを渡すことによって
call by referenceをシミュレートできる。

```
void swap (int * px, int * py) {  
    int z;  
    z = *px; *px = *py; *py = z;  
}
```

この関数swapを呼び出す際、ポインタ(アドレス)を渡す。
。

```
int a = 1, b = 2;  
swap (&a, &b);
```

これにより、変数aとbの値が入れ替わる。

Call by name

Call by nameでは、手続きの本体の中の仮引数を実引数の式で置き換えて実行する。ただし、この置き換えで名前の衝突が起こる場合は、名前の付け替えを行う。Algol60はデフォルトでcall by nameである。
(Algol68ではcall by nameはなくなった。)

```
program {内積の計算}
```

```
  var i, n, z : integer;
```

```
    a, b : array [0..9] of integer;
```

```
  procedure sum (x : integer);
```

```
    begin while i < n do begin z := z + x; i := i + 1 end end;
```

```
  begin n := 10; i := 0; z := 0; sum (a[i] * b[i]); writeln (z) end.
```

引数は手続き内部で仮引数が現れた箇所で評価される。これを遅延評価という。遅延評価にはcall by nameとcall by need(引数を一度だけ評価)がある。副作用が無い場合これら2つの評価は一致する。

(補足) Call by value result

Call by referenceと似たような効果を持つものとして Call by value resultがある。これは、仮引数に実引数の値を渡し、実引数のアドレスを呼び出し時に保存しておいて、手続き終了時の仮引数の値を、保存しておいたアドレスに代入するというものである。Copy-in/copy-outとも呼ばれる。Adaではin, out, in outという3つの引数渡し方法が使える。In outがcall by value resultである。Call by value resultはcall by referenceと似ているが、異なる。

(例) **program**

i, j : integer;

procedure foo (x, y : integer); begin i := y end;

begin i := 2; j := 3; foo (i, j) end

Call by referenceではiの値は3になるが、call by value resultでは呼び出し後、iの値は2のままである。

変数の有効範囲について

変数には有効範囲(scope)がある。有効範囲の定め方は、静的有効範囲(static scopeあるいはlexical scope)、動的有効範囲(dynamic scope)に分かれる。

プログラム中の各変数がどこで宣言された変数であるかの決め方をスコープ規則(scope rule)という。スコープ規則を定めることにより、各宣言で宣言される名前の有効範囲が定まる。

静的有効範囲では、プログラムテキスト上で各宣言で宣言される変数の有効範囲が分かる。動的有効範囲では、変数の有効範囲は実行状況に依存する。

プログラム例

program L;	{続き(プログラムLの本体)}
var n : char;	begin
procedure W;	n := 'L';
begin	W;
writeln(n)	D
end;	end.
procedure D;	
var n : char;	
begin	
n := 'D';	
W	
end;	

Static scope

Static scopeでは、プログラム中のある変数 x の宣言は、プログラムテキスト上で、その変数を含む一番内側にある、 x の宣言である。

Static scopeはlexical scopeとも呼ばれる。コンパイル時に変数の有効範囲が分かる。PascalやC等、ほとんどの言語はstatic scopeである。さきほどのプログラム例の実行結果は、static scopeでは

L

L

となる。

(考え方) Localな変数は名前を付け替えても意味は変わらないというのは自然な考え方である。この考えに従うとstatic scopeになる。

Dynamic scope

Dynamic scopeでは、手続きpの呼び出し時点で、ある変数xが有効でxに対応する宣言がdの場合、pが呼ばれても変数に対応する宣言はdである。宣言dがなされている手続きが終了すると、xとdとの関係はなくなる。ただし、変数xが有効な状況で変数xの宣言d'がなされた場合、そこからはその変数xに対応する宣言はd'となる。

Emacs Lispはdynamic scopeである。さきほどのプログラム例の実行結果はdynamic scopeだと

L

D

になる。

(考え方) Dynamic scopeでは、ある変数xは、スタック上で一番近いactivation recordのxに対応する。